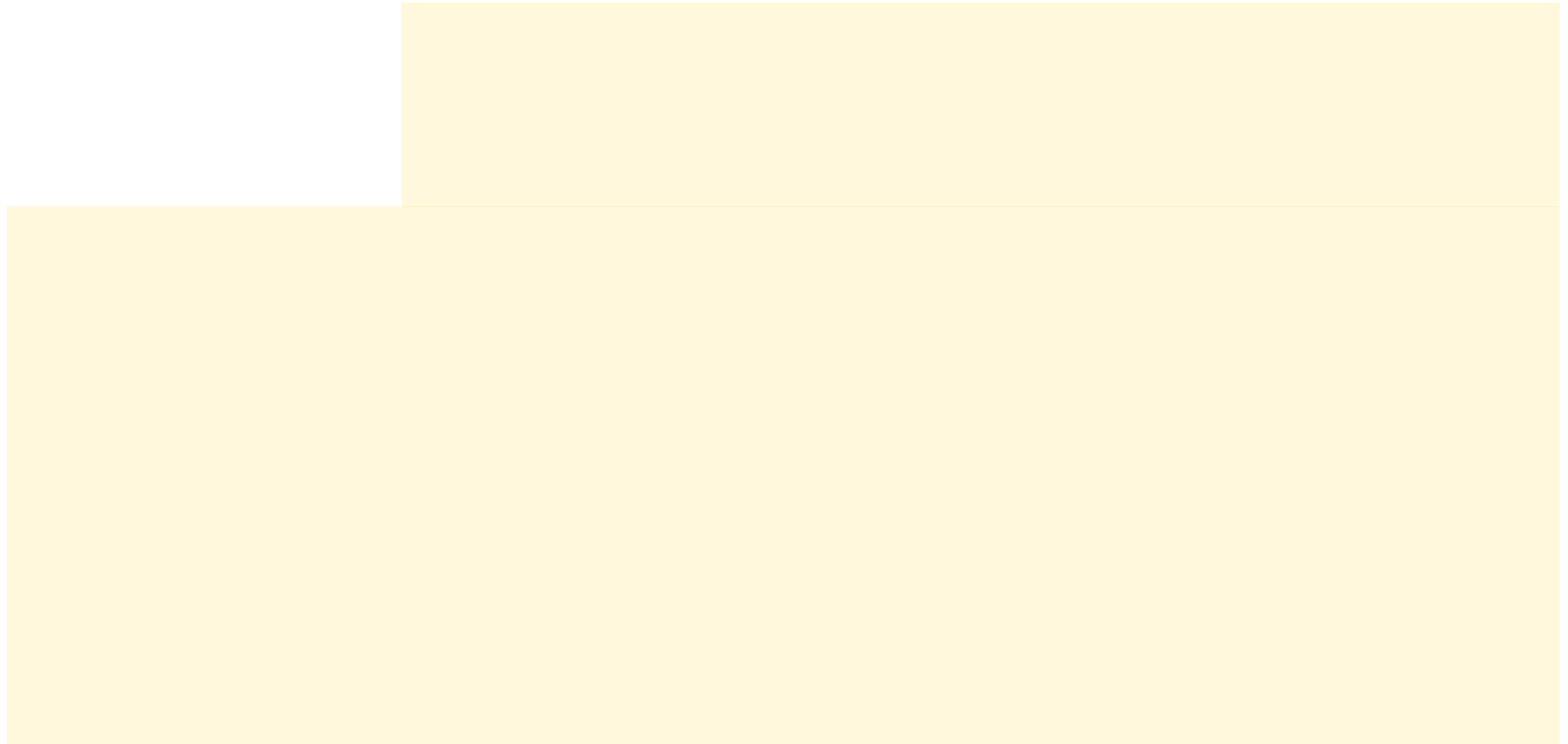


Participando del Espacio de Entrada

Valeria Bengolea

Estas transparencias están basadas en las desarrolladas por Ammann & Offutt
como acompañamiento de su libro Introduction to Software Testing (2nd Edition) y
por Manuel Núñez (Versión en castellano)

Cuatro estructuras para modelar software



Cuatro estructuras para modelar software

```
graph TD; A[Cuatro estructuras para modelar software] --- B[Espacio de inputs]; A --- C[Grafos]; A --- D[Lógica]; A --- E[Syntaxis];
```

Espacio de
inputs

Grafos

Lógica

Syntaxis

Cuatro estructuras para modelar software

Espacio de inputs

Grafos

Lógica

Syntaxis

Aplicado a

Código

Especs.

Diseño

Casos de uso

Cuatro estructuras para modelar software

Espacio de inputs

Grafos

Lógica

Syntaxis

Aplicado a

Aplicado a

Código

Especs.

Diseño

Casos de uso

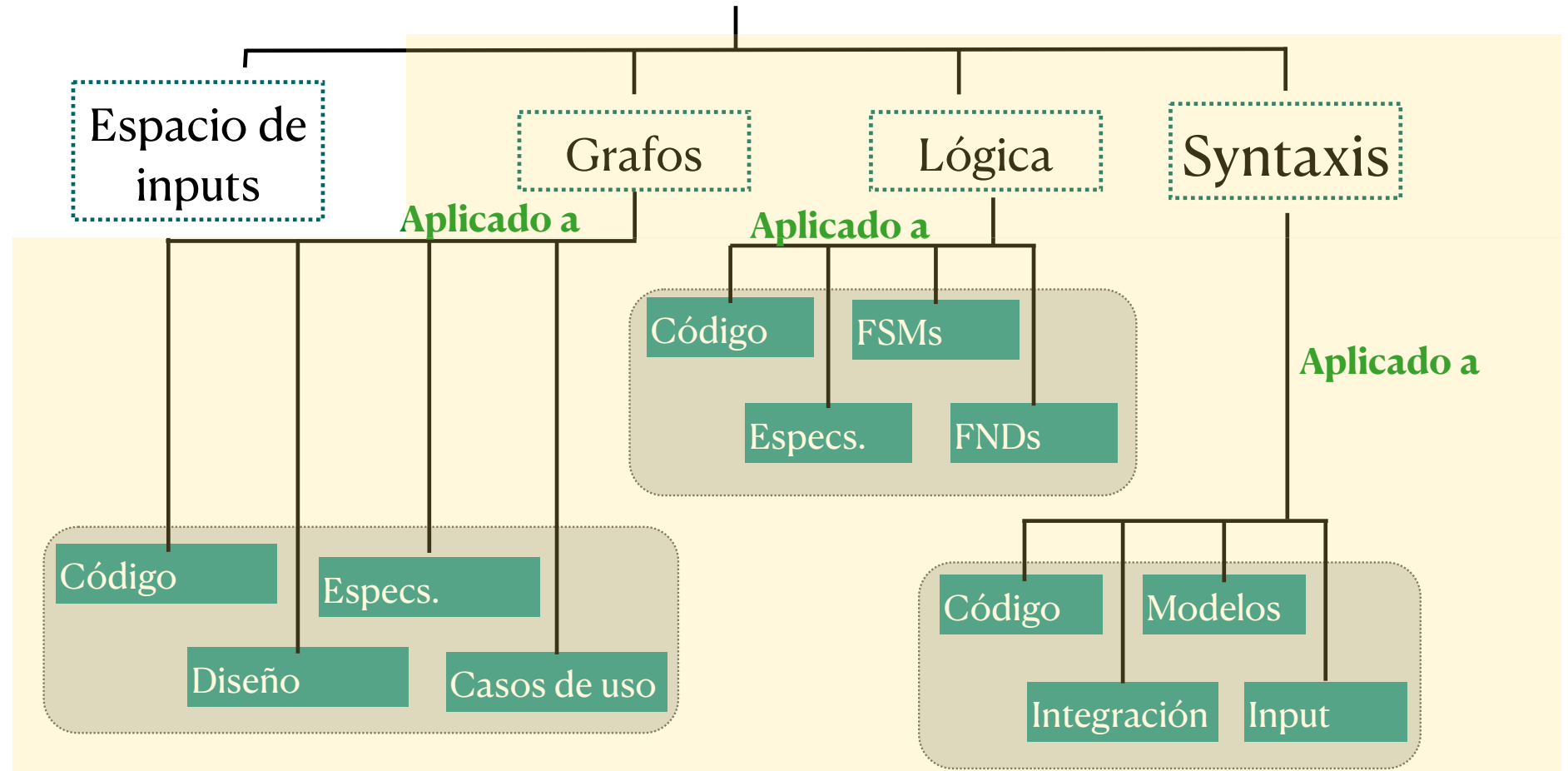
Código

FSMs

Especs.

FNDs

Cuatro estructuras para modelar software



Partición del Espacio de Inputs

La partición del espacio de entrada considera que podemos dividir directamente el espacio de entrada de acuerdo con particiones lógicas de las entradas.

Dominio de Entradas

El **dominio de entradas** de un programa contiene todos los posibles inputs del programa.

★ Incluso para programas pequeños, el dominio de inputs es tan grande que podría considerarse **infinito**.

Problema: **elegir un conjunto finito** de valores de este dominio que encuentren la mayoría de los defectos del código

Dominio de Entradas

Los **parámetros de entrada** definen el alcance del dominio de las entradas:

- ★ Valores para parámetros de métodos
- ★ Datos en archivos
- ★ Valores de variables globales
- ★ Entradas pedidas al usuario
- ★ Etc.

Los dominios asociados con los parámetros de entrada se **particionan en bloques**

Se escoge **al menos un valor** de cada uno de estos bloques

Dominio de Entradas

Los **parámetros de entrada** definen el alcance del dominio de las entradas:

- ★ Valores para parámetros de métodos
- ★ Datos en archivos
- ★ Valores de variables globales
- ★ Entradas pedidas al usuario
- ★ Etc.

Dominio de Entradas: Letras alfabéticas
Característica de particionado: Tipo de Letra
(Mayúscula/Minúscula)

- ★Bloque 1: Mayúscula
- ★Bloque 2: Minúscula

Los dominios asociados con los parámetros de entrada se **particionan en bloques**

Se escoge **al menos un valor** de cada uno de estos bloques

Particionado del espacio de entradas

Usando la especificación del SUT buscar **características** que se consideran importantes para testing

Ejemplos:

- Rutina que computa el valor absoluto de un entero n
 - **Características:** “relación de n con cero”
 - **Bloques:** $n = 0$, $n > 0$ y $n < 0$
- Rutina que calcula si un año es bisiesto o no:
 - **Característica:** “Año bisiesto”
 - **Bloques:** true/false

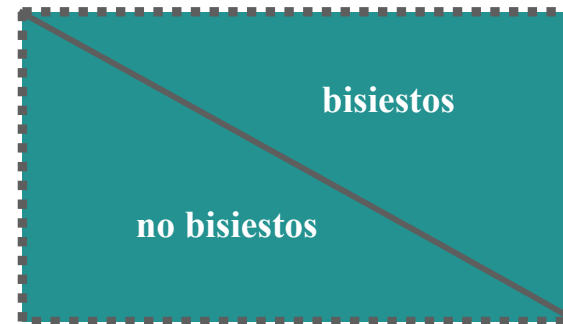
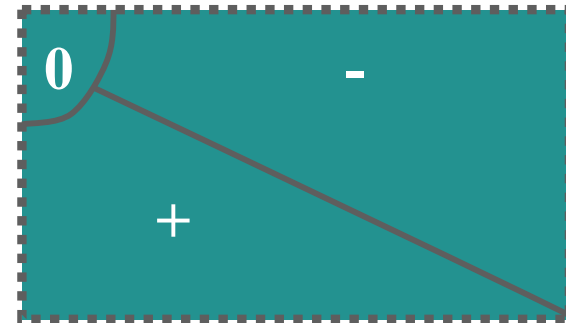
Particionar el conjunto de entradas en bloques de acuerdo a las características elegidas

Particionado del espacio de entradas

- Rutina que computa el valor absoluto de un entero n :
Características “relación de n con cero”: $n = 0$, $n > 0$ y $n < 0$
- Rutina que calcula si un año es bisiesto o no: Característica
“Año bisiesto”: true/false

Particionado del espacio de entradas

- Rutina que computa el valor absoluto de un entero n :
Características “relación de n con cero”: $n = 0$, $n > 0$ y $n < 0$
- Rutina que calcula si un año es bisiesto o no: Característica
“Año bisiesto”: true/false



Particionado del Dominio

Dominio D

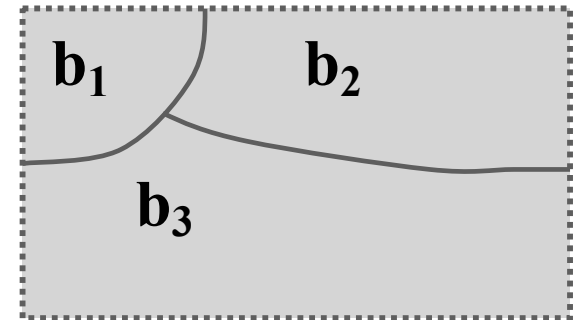
Dominio de las
entradas: Contiene todas las
posibles entradas de un
programa

La partición q define un conjunto de bloques, $B_q = b_1, b_2, \dots, b_n$

La partición debe satisfacer 2 propiedades:

1. Los bloques deben ser disjuntos de a pares:

$$b_i \cap b_j = \Phi, \forall i \neq j, b_i, b_j \in B_q$$



2. Los bloques juntos deben cubrir todo el dominio (completitud)

$$\bigcup_{b \in B_q} b = D$$

Particionado de

Valores en un mismo bloque se asumen "igualmente útiles" para testing

Elegir un valor por cada bloque para construir los tests

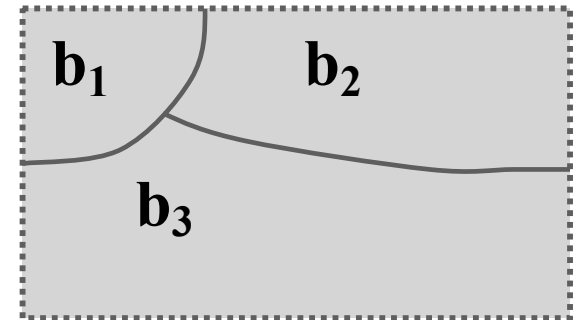
Dominio D

La partición q define un conjunto de bloques, $B_q = b_1, b_2, \dots, b_n$

La partición debe satisfacer 2 propiedades:

1. Los bloques deben ser disjuntos de a pares:

$$b_i \cap b_j = \Phi, \forall i \neq j, b_i, b_j \in B_q$$



2. Los bloques juntos deben cubrir todo el dominio (completitud)

$$\bigcup_{b \in B_q} b = D$$

Beneficios de Particionado de Espacio de Entradas

Se puede aplicar a varios niveles de testing

Unitario.

Integración.

Sistema.

Es fácil adaptar el proceso para considerar más/menos tests.

No se necesita conocer la implementación: simplemente el espacio de inputs.

¿Cómo definimos las particiones?

Identificaremos características importantes de las entradas, que inducen particiones en los dominios

Las características son escenarios deseados de testing

Ejemplos de características:

- Relación de X respecto a null (igual o distinto de null)
- Orden de archivo F (ordenado de menor a mayor, ordenado de mayor a menor, orden arbitrario, ...)
- Dispositivo de entrada conectado en un puerto (Flash drive, disco externo, BluRay, ...)

¿Cómo definimos las particiones?

Usualmente, las características se definen en base a **especificaciones**

- ★ Luego del particionado, se elige un valor de cada bloque como entrada para test
 - ★ Valores dentro de un mismo bloque se asumen igualmente útiles para testing
- ★ Tener en cuenta:
 - ★ Precondiciones y poscondiciones del método
 - ★ Relaciones entre variables
 - ★ Relaciones de variables con valores especiales (cero, Null, espacios en blanco, true/false,...)
- ★ No usar el código del programa, basarse en especificaciones

Definiendo particiones

Definir particiones parece sencillo, pero es fácil cometer errores

Considere la característica “orden del archivo F”

Bloques:

b_1 : orden creciente

b_2 : orden decreciente

b_3 : orden arbitrario

Definiendo particiones

Definir particiones parece sencillo, pero es fácil cometer errores

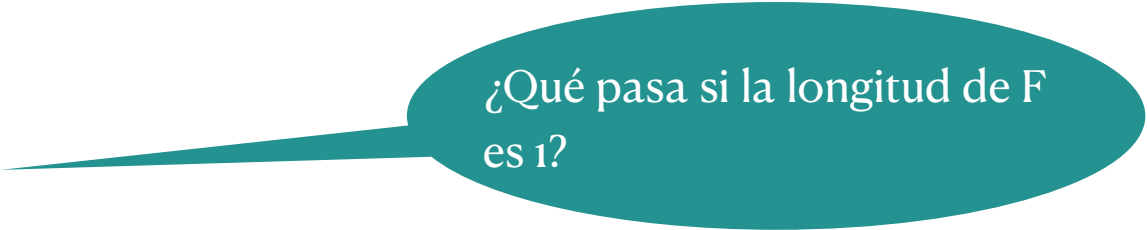
Considere la característica “orden del archivo F”

Bloques:

b_1 : orden creciente

b_2 : orden decreciente

b_3 : orden arbitrario



¿Qué pasa si la longitud de F es 1?

Definiendo particiones

Definir particiones parece sencillo, pero es fácil cometer errores

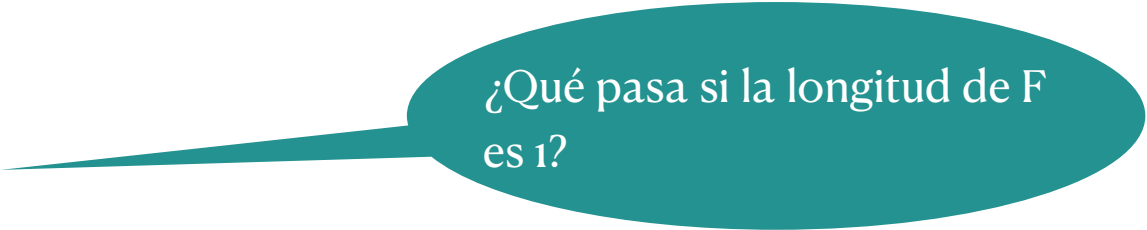
Considere la característica “orden del archivo F”

Bloques:

b_1 : orden creciente

b_2 : orden decreciente

b_3 : orden arbitrario



¿Qué pasa si la longitud de F es 1?

El archivo estará en los 3 bloques: **hay solapamiento**

Propiedades de las particiones

Si las particiones no son completas o disjuntas, no han sido definidas correctamente.

- ★ Si una entrada cubre dos bloques, o hay entradas que no pertenecen a ningún bloque, se están ignorando partes del dominio que podrían causar **fallas**.
- ★ En estos casos, las particiones deben ser cuidadosamente revisadas, y deben considerarse modificaciones.

Definición de particiones: Ejemplo

Solución: Definir dos características, cada una debería considerar solo una propiedad

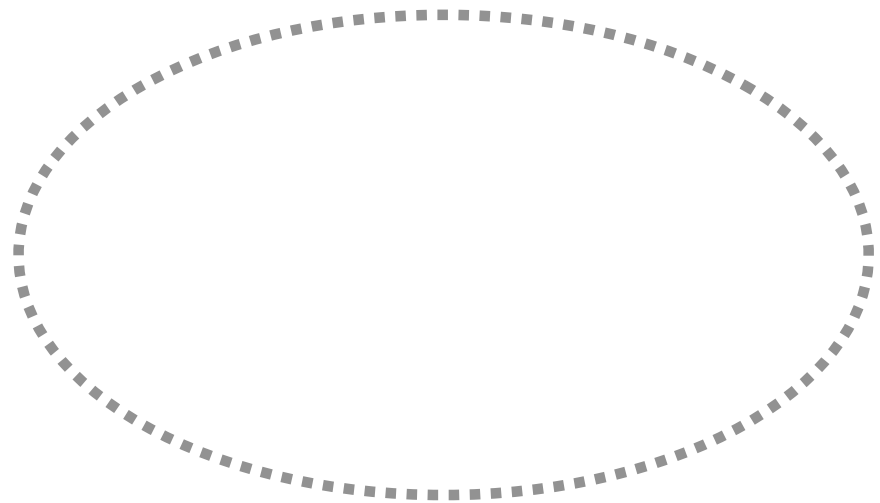
Características : C1 y C2

C1 : orden creciente

- c1.b1:true
- c1.b2:false

C2 : orden decreciente

- c2.b1:true
- c2.b2:false



Definición de particiones: Ejemplo

Solución: Definir dos características, cada una debería considerar solo una propiedad

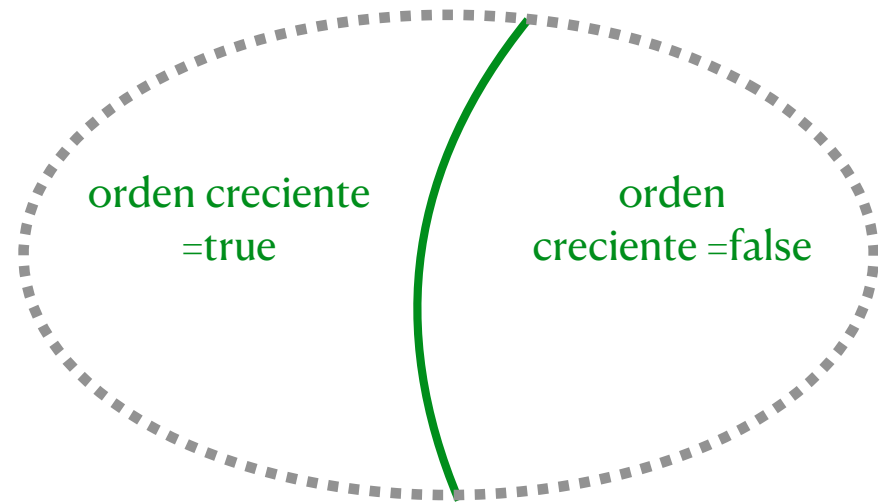
Características : C1 y C2

C1 : orden creciente

- c1.b1:true
- c1.b2:false

C2 : orden decreciente

- c2.b1:true
- c2.b2:false



Definición de particiones: Ejemplo

Solución: Definir dos características, cada una debería considerar solo una propiedad

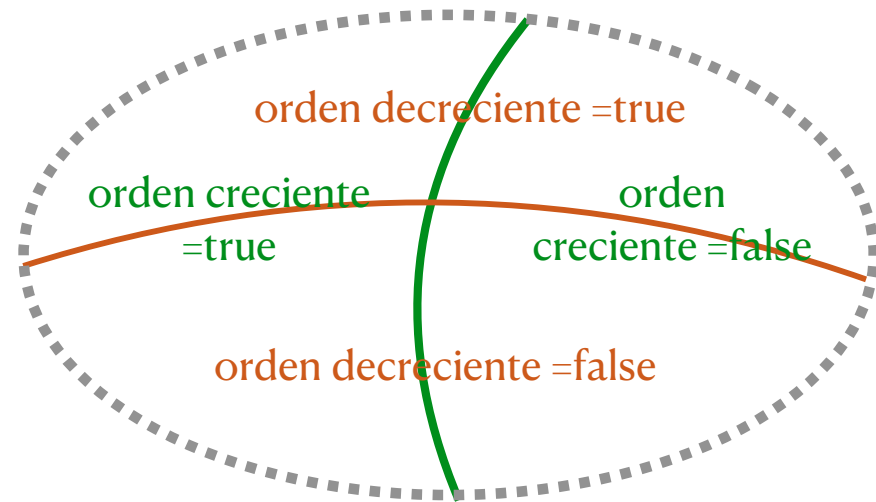
Características : C1 y C2

C1 : orden creciente

- c1.b1:true
- c1.b2:false

C2 : orden decreciente

- c2.b1:true
- c2.b2:false



Modelado del Dominio de las entradas

Paso 1: Identificar funcionalidad a testear

- Testing de unidad: Métodos individuales
- Testing de integración: Métodos de una o varias clases
- Testing de sistema: Programa

Paso 2: Listar **todas** las entradas

- Métodos: Parámetros y estado (variables no locales)
- Módulo: Parámetros de métodos y variables globales involucrados
- Sistema: Todas las entradas (incl. archivos, bases de datos,...)

Modelado del Dominio de las entradas

Paso 1: Identificar funcionalidad a testear

- Testing de unidad: Métodos individuales
- Testing de integración: Métodos de una o varias clases
- Testing de sistema: Programa

Paso 2: Listar **todas** las entradas

- Métodos: Parámetros y estado (variables no locales)
- Módulo: Parámetros de métodos y variables globales involucrados
- Sistema: Todas las entradas (incl. archivos, bases de datos,...)



Nivel de
abstracción
CONCRETO

Modelado del Dominio de las entradas

Paso 3: Crear un modelo del dominio de las entradas

- La estructura se define en términos de características
- Cada característica se particiona en conjuntos de bloques
- Cada bloque representa un conjunto de valores

Paso 4: Aplicar un criterio para crear combinaciones de bloques

Paso 5: Obtener entradas para testing a partir de las combinaciones de bloques

Un Modelo del Dominio de Entradas (*MDE*) representa el espacio de entradas de un SUT de una forma abstracta

Modelado del Dominio de las entradas

Paso 3: Crear un modelo del dominio de las entradas

- La estructura se define en términos de características
- Cada característica se particiona en conjuntos de bloques
- Cada bloque representa un conjunto de valores

Paso 4: Aplicar un criterio para crear combinaciones de bloques

Paso 5: Obtener entradas para testing a partir de las combinaciones de bloques



Nos movemos del nivel de abstracción de la implementación al nivel de abstracción de diseño

Un Modelo del Dominio de Entradas (*MDE*) representa el espacio de entradas de un SUT de una forma abstracta

Modelado del Dominio de las entradas

Paso 3: Crear un modelo del dominio de las entradas

- La estructura se define en términos de características
- Cada característica se particiona en conjuntos de bloques
- Cada bloque representa un conjunto de valores

Paso 4: Aplicar un criterio para crear combinaciones de bloques

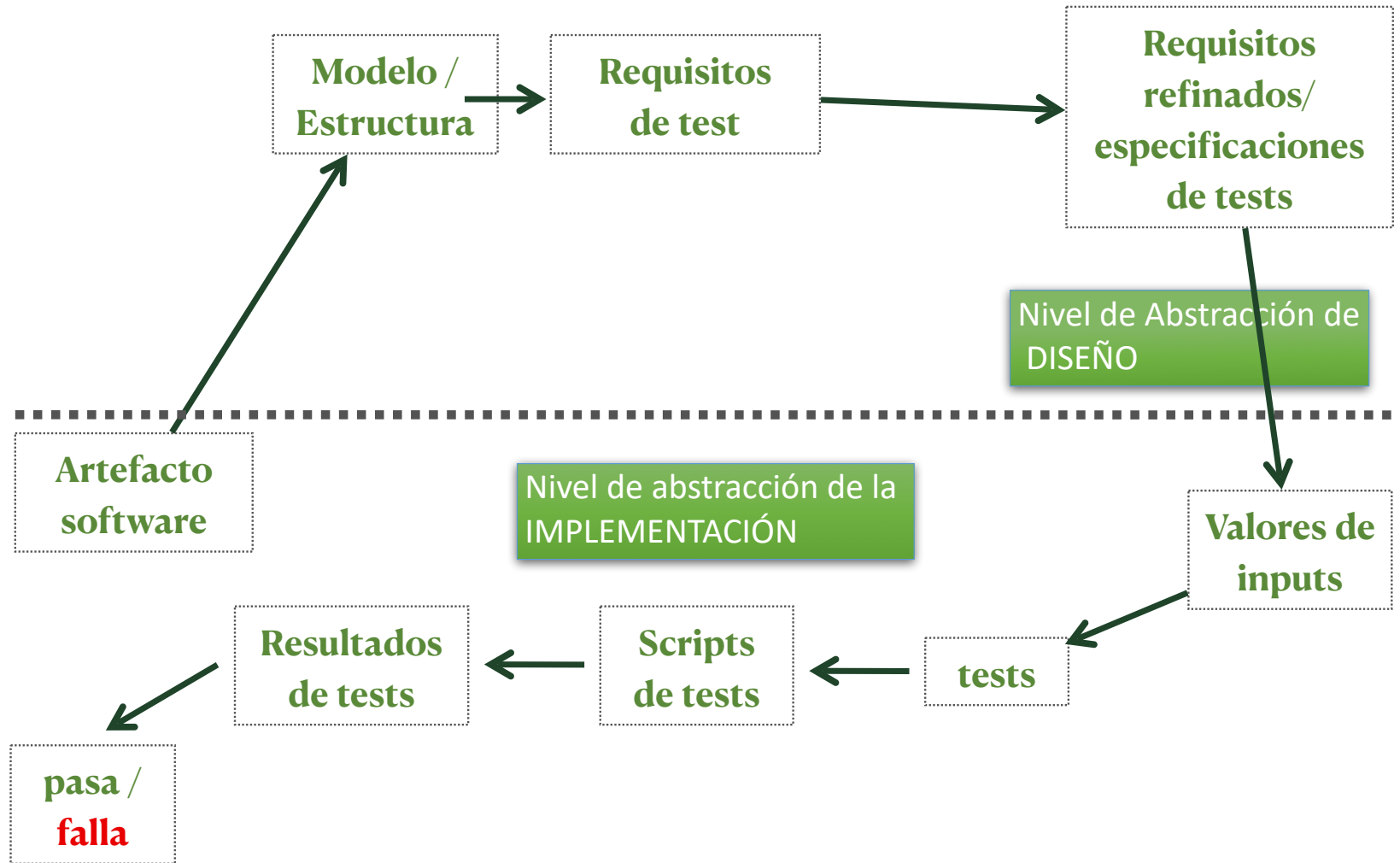
Paso 5: Obtener entradas para testing a partir de las combinaciones de bloques

Nos movemos del nivel de abstracción de la implementación al nivel de abstracción de diseño

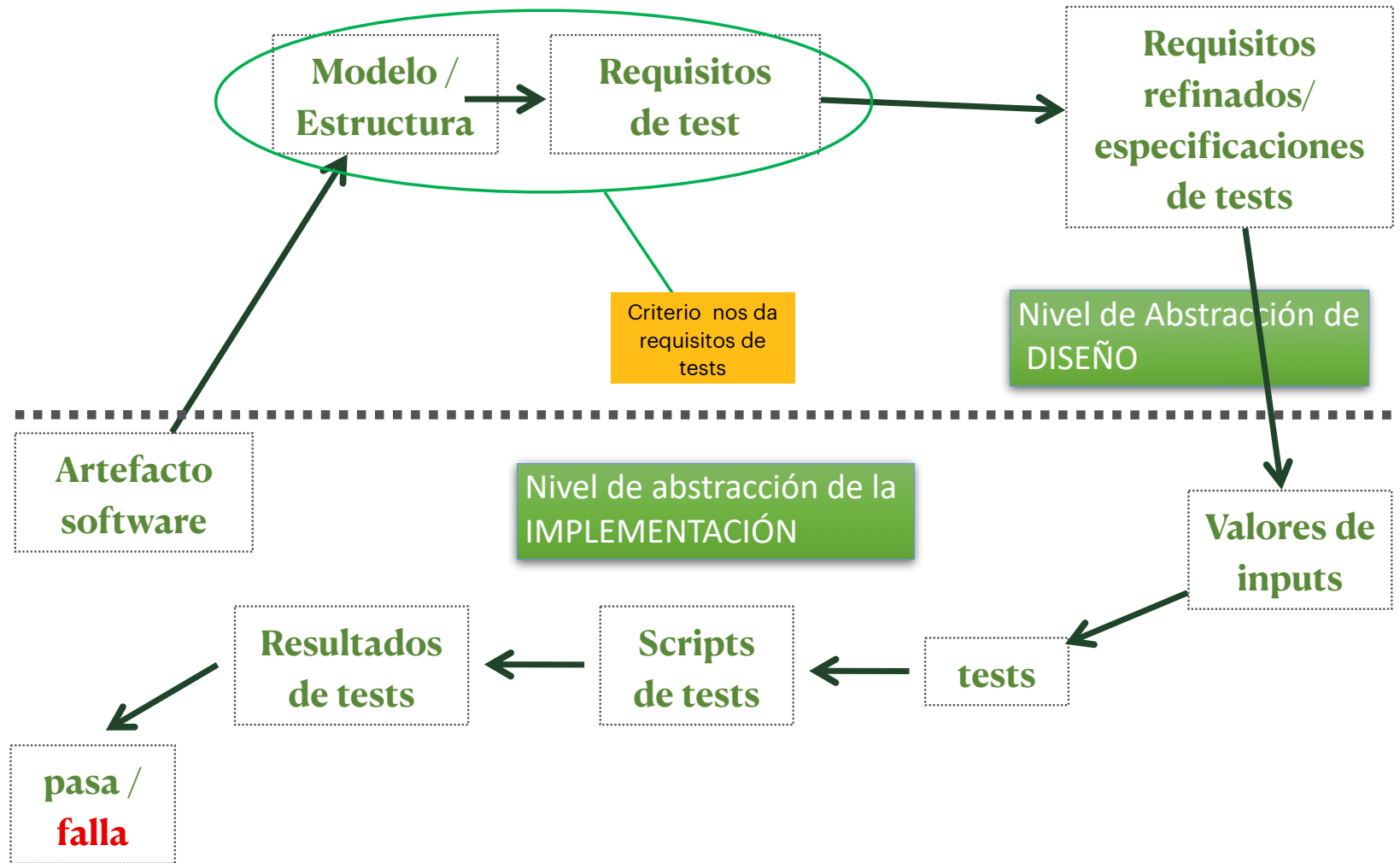
Volvemos al nivel de abstracción de la implementación

Un Modelo del Dominio de Entradas (*MDE*) representa el espacio de entradas de un SUT de una forma abstracta

Model-Driven Test Diseño



Model-Driven Test Diseño



Modelado del Dominio de las entradas: Dos enfoques

Enfoque basado en la **interfaz**:

- ★ Desarrollar características a partir de parámetros individuales de manera aislada (con respecto al resto de los parámetros)
- ★ Basado en la sintaxis
- ★ Información semántica y del dominio podría no tenerse en cuenta
- ★ Ignora las relaciones entre parámetros

Modelado del Dominio de las entradas: Dos enfoques

Enfoque basado en la **funcionalidad**:

- ★ Desarrollar características que se corresponden con el comportamiento esperado del programa bajo test.
- ★ Permite incorporar información semántica y del dominio.
- ★ Incorpora relaciones entre los parámetros
- ★ Requiere más esfuerzo de diseño.

Modelo del Dominio de las Entradas (MDE)

Diseño de Características

- ★ Diseñar características en un enfoque basado en interfaz es simple.
- ★ Desarrollar un MDE basado en funcionalidades es más desafiante.
 - ★ Las precondiciones son excelentes fuentes de características. Separan explícitamente el comportamiento normal del comportamiento excepcional.
 - ★ Las poscondiciones también son buenas fuentes de características. Para el método `kindOfTriangle()`, los diferentes tipos de triángulos se basan en la poscondición del método.
 - ★ También se deben buscar otras relaciones entre variables. Estos pueden ser explícitas o implícitas.
- ★ La idea es que el tester debería aplicar el modelado del dominio de la entrada utilizando el conocimiento del dominio sobre el problema, no la implementación.

MDE basado en la interfaz: Un ejemplo

Considere el método `kindOfTriangle()`

```
public enum TriangleType {Equilateral, Isosceles, Scalene};
```

```
//Side1, Side2, and Side3 represent the lengths of the sides of a triangle  
//Returns the appropriate enum value
```

```
public TriangleType kindOfTriangle(int side1, int side2, int side3){  
    ...  
}
```

MDE basado en la interfaz: Un ejemplo

Considere el método `kindOfTriangle()`

```
public enum TriangleType {Equilateral, Isosceles, Scalene};

//Side1, Side2, and Side3 represent the lengths of the sides of a triangle
//Returns the appropriate enum value

public TriangleType kindOfTriangle(int side1, int side2, int side3){
    ...
}
```

Una Característica para cada parámetro.

Características: Relación del lado (parámetro) respecto a cero

Bloques: (> 0 , $= 0$, < 0)

MDE basado en la interfaz: Un ejemplo

Primer modelo

Modelo de entradas

Característica	b_1	b_2	b_3
q_1 = “Relación de Lado 1 a o”	Mayor a o	Igual a o	Menor a o
q_2 = “Relación de Lado 2 a o”	Mayor a o	Igual a o	Menor a o
q_3 = “Relación de Lado 3 a o”	Mayor a o	Igual a o	Menor a o

Un **máximo** de $3 \times 3 \times 3 = 27$ test
Muchos triángulos son inválidos

MDE basado en la interfaz: Un ejemplo

Modelo refinado

Modelo de entradas

Característica	b_1	b_2	b_3	b_4
$c_1 = \text{Refina } q_1$	mayor a 1	igual a 1	igual a 0	menor a 0
$c_2 = \text{Refina } q_2$	mayor a 1	igual a 1	igual a 0	menor a 0
$c_3 = \text{Refina } q_3$	mayor a 1	igual a 1	igual a 0	menor a 0

MDE basado en la interfaz: Un ejemplo

Modelo refinado

Modelo de entradas

Característica	b_1	b_2	b_3	b_4
$c_1 = \text{Refina } q_1$	mayor a 1	igual a 1	igual a 0	menor a 0
$c_2 = \text{Refina } q_2$	mayor a 1	igual a 1	igual a 0	menor a 0
$c_3 = \text{Refina } q_3$	mayor a 1	igual a 1	igual a 0	menor a 0

MDE basado en la interfaz: Un ejemplo

Modelo refinado

Modelo de entradas

Característica	b_1	b_2	b_3	b_4
$c_1 = \text{Refina } q_1$	mayor a 1	igual a 1	igual a 0	menor a 0
$c_2 = \text{Refina } q_2$	mayor a 1	igual a 1	igual a 0	menor a 0
$c_3 = \text{Refina } q_3$	mayor a 1	igual a 1	igual a 0	menor a 0

Da lugar a un máximo de $4 \times 4 \times 4 = 64$ test
Completa porque los inputs son enteros

MDE basado en la interfaz: Un ejemplo

Característica	b_1	b_2	b_3	b_4
$c_1 = \text{Refina } q_1$	mayor a 1	igual a 1	igual a 0	menor a 0
$c_2 = \text{Refina } q_2$	mayor a 1	igual a 1	igual a 0	menor a 0
$c_3 = \text{Refina } q_3$	mayor a 1	igual a 1	igual a 0	menor a 0

Valores posibles para q_1

Característica	b_1	b_2	b_3	b_4
Lado1	5	1	0	-5

MDE basado en la interfaz: Un ejemplo

Característica	b_1	b_2	b_3	b_4
$c_1 = \text{Refina } q_1$	mayor a 1	igual a 1	igual a 0	menor a 0
$c_2 = \text{Refina } q_2$	mayor a 1	igual a 1	igual a 0	menor a 0
$c_3 = \text{Refina } q_3$	mayor a 1	igual a 1	igual a 0	menor a 0

Valores posibles para q_1

Característica	b_1	b_2	b_3	b_4
Lado1	2	1	0	-1

MDE basado en la interfaz: Un ejemplo

Característica	b_1	b_2	b_3	b_4
$c_1 = \text{Refina } q_1$	mayor a 1	igual a 1	igual a 0	menor a 0
$c_2 = \text{Refina } q_2$	mayor a 1	igual a 1	igual a 0	menor a 0
$c_3 = \text{Refina } q_3$	mayor a 1	igual a 1	igual a 0	menor a 0

Valores posibles para q_1

Característica	b_1	b_2	b_3	b_4
Lado1	2	1	0	-1

Tener en cuenta Valores Bordes!

MDE basado en la funcionalidad

Un ejemplo

Los dos MDE anteriores están basadas en la sintaxis: parámetros y tipos.

Una característica a nivel semántico:

Característica	b_1	b_2	b_3	b_4
q_1 = Clasificación Geométrica	escaleno	isósceles	equilátero	inválido

MDE basado en la funcionalidad

Un ejemplo

Los dos MDE anteriores están basadas en la sintaxis: parámetros y tipos

Una característica a nivel semántico:

Característica	b_1	b_2	b_3	b_4
q_1 = Clasificación Geométrica	escaleno	isósceles	equilátero	inválido

problema: los equiláteros también son isósceles

MDE basado en la funcionalidad

Un ejemplo

Los dos MDE anteriores están basadas en la sintaxis: parámetros y tipos

Una característica a nivel semántico:

problema: los equiláteros
también son isósceles

Característica	b_1	b_2	b_3	b_4
q_1 = Clasificación Geométrica	escaleno	isósceles, no equiláteros	equilátero	inválido

MDE basado en la funcionalidad

Un ejemplo

Posibles valores para partición q_1 :

Característica	b_1	b_2	b_3	b_4
Triángulo	(4,5,6)	(3,3,4)	(3,3,3)	(3,4,8)

MDE basado en la funcionalidad

Un ejemplo

Posibles valores para partición q_1 :

(a, b, c) forman un triángulo válido si:
 $a+b>c \ \&\& \ a+c>b \ \&\& \ b+c>a$

Característica	b_1	b_2	b_3	b_4
Triángulo	(4,5,6)	(3,3,4)	(3,3,3)	(3,4,8)

MDE basado en la funcionalidad

Un ejemplo

Otra alternativa que involucra más de una característica:

Característica	b ₁	b ₂
q ₁ = escaleno	True	False
q ₂ = isósceles	True	False
q ₃ = equilátero	True	False
q ₄ = válido	True	False

Imponer restricciones sobre este Modelo:

equilátero = True ==> isósceles = true

válido = False ==> escaleno = isósceles = equilatero = False

Ejemplo: Enfoques basados en interfaz y funcionalidad

```
public int numberOfOccurrences(List, element);  
//Effects: if list or element is Null throws ValueError  
//else return number of occurrences of element in List
```

Ejemplo: Enfoques basados en interfaz y funcionalidad

```
public int numberOfOccurrences(List, element);  
//Effects: if list or element is Null throws ValueError  
//else return number of occurrences of element in List
```

Enfoque basado en la interfaz

Dos parámetros: List, element

Características:

list es Null (b1 = true, b2 = false)

list es vacía (b1 = true, b2 = false)

Ejemplo: Enfoques basados en interfaz y funcionalidad

```
public int numberOfOccurrences(List, element);  
//Effects: if list or element is Null throws ValueError  
//else return number of occurrences of element in List
```

Enfoque basado en la interfaz

Dos parámetros: List, element

Características:

list es Null (b1 = true, b2 = false)

list es vacía (b1 = true, b2 = false)

Enfoque basado en la funcionalidad

Dos parámetros: list, element

Algunas Características:

número de ocurrencias de element en list (0, 1, >1)

element está primero en list (true, false)

element está último en list (true, false)

Particiones y valores de test

- ★ Los tests surgen de elegir valores para los bloques de las distintas características
 - ★ Más bloques implican más tests
- ★ Estrategias para crear particiones y elegir valores:
 - ★ Incluir valores que representan “uso normal”
 - ★ Incluir valores inválidos y especiales
 - ★ Considerar valores límites
 - ★ Volver a particionar algunos bloques de ser necesario
 - ★ Tratar de balancear el número de bloques en las distintas características
 - ★ Verificar que las particiones sean completas y disjuntas
- ★ Generalmente es preferible tener más características con pocos bloques que lo contrario
 - ★ Menos errores en las particiones

¿Cómo elegir combinaciones de valores?

- ★ Una vez definidas las características y particiones, debemos **combinar bloques de distintas características** y crear requerimientos de tests
- ★ Usaremos **criterios** para crear requisitos de tests (y tests)
- ★ Varios criterios para elegir combinaciones:
 - ★ El más obvio es hacer todas las combinaciones de bloques

Criterio: Todas las Combinaciones

- Llamado también **ACoC** por sus siglas en inglés “All combination Coverage” Se usan todas las combinaciones de bloques de todas las características.
 - El número de tests es igual al producto del número de bloques en cada característica.

ACoC: Ejemplo

```
public TriangleType kindOfTriangle(int side1, int side2, int side3);
```

MDE:

Característica	b ₁	b ₂	b ₃	b ₄
c ₁	mayor a 1	igual a 1	igual a 0	menor a 0
c ₂	mayor a 1	igual a 1	igual a 0	menor a 0
c ₃	mayor a 1	igual a 1	igual a 0	menor a 0

re-etiquetamos los bloques (por comodidad)

Característica	b ₁	b ₂	b ₃	b ₄
A	A1	A2	A3	A4
B	B1	B2	B3	B4
C	C1	C2	C3	C4

Requisitos de Test

ACoC

A1 B1 C1 A2 B1 C1 A3 B1 C1 A4 B1 C1
A1 B1 C2 A2 B1 C2 A3 B1 C2 A4 B1 C2
A1 B1 C3 A2 B1 C3 A3 B1 C3 A4 B1 C3
A1 B1 C4 A2 B1 C4 A3 B1 C4 A4 B1 C4

A1 B2 C1 A2 B2 C1 A3 B2 C1 A4 B2 C1
A1 B2 C2 A2 B2 C2 A3 B2 C2 A4 B2 C2
A1 B2 C3 A2 B2 C3 A3 B2 C3 A4 B2 C3
A1 B2 C4 A2 B2 C4 A3 B2 C4 A4 B2 C4

A1 B3 C1 A2 B3 C1 A3 B3 C1 A4 B3 C1
A1 B3 C2 A2 B3 C2 A3 B3 C2 A4 B3 C2
A1 B3 C3 A2 B3 C3 A3 B3 C3 A4 B3 C3
A1 B3 C4 A2 B3 C4 A3 B3 C4 A4 B3 C4

A1 B4 C1 A2 B4 C1 A3 B4 C1 A4 B4 C1
A1 B4 C2 A2 B4 C2 A3 B4 C2 A4 B4 C2
A1 B4 C3 A2 B4 C3 A3 B4 C3 A4 B4 C3
A1 B4 C4 A2 B4 C4 A3 B4 C4 A4 B4 C4

Característica	b ₁	b ₂	b ₃	b ₄
A	A1	A2	A3	A4
B	B1	B2	B3	B4
C	C1	C2	C3	C4

A1 B1 C1: elegir una entrada (y construir un test)
que cumpla con:
`lado1 > 1 && lado2 > 1 && lado3 > 1`

Requisitos de Test

ACoC

A1 B1 C1 A2 B1 C1 A3 B1 C1 A4 B1 C1
 A1 B1 C2 A2 B1 C2 A3 B1 C2 A4 B1 C2
 A1 B1 C3 A2 B1 C3 A3 B1 C3 A4 B1 C3
 A1 B1 C4 A2 B1 C4 A3 B1 C4 A4 B1 C4

 A1 B2 C1 A2 B2 C1 A3 B2 C1 A4 B2 C1
 A1 B2 C2 A2 B2 C2 A3 B2 C2 A4 B2 C2
 A1 B2 C3 A2 B2 C3 A3 B2 C3 A4 B2 C3
 A1 B2 C4 A2 B2 C4 A3 B2 C4 A4 B2 C4

 A1 B3 C1 A2 B3 C1 A3 B3 C1 A4 B3 C1
 A1 B3 C2 A2 B3 C2 A3 B3 C2 A4 B3 C2
 A1 B3 C3 A2 B3 C3 A3 B3 C3 A4 B3 C3
 A1 B3 C4 A2 B3 C4 A3 B3 C4 A4 B3 C4

 A1 B4 C1 A2 B4 C1 A3 B4 C1 A4 B4 C1
 A1 B4 C2 A2 B4 C2 A3 B4 C2 A4 B4 C2
 A1 B4 C3 A2 B4 C3 A3 B4 C3 A4 B4 C3
 A1 B4 C4 A2 B4 C4 A3 B4 C4 A4 B4 C4

Característica	b ₁	b ₂	b ₃	b ₄
A	A1	A2	A3	A4
B	B1	B2	B3	B4
C	C1	C2	C3	C4

A1 B1 C1: elegir una entrada (y construir un test)
 que cumpla con:
 $\text{lado1} > 1 \ \&\& \ \text{lado2} > 1 \ \&\& \ \text{lado3} > 1$

ACoC da lugar a $4 \times 4 \times 4 = 64$ tests.

Ciertamente, esta cantidad parece muy alta.

En particular, teniendo en cuenta que solo 8 de ellos son válidos (todos los lados mayores que cero).

Each Choice Coverage (ECC)

La idea es que **cada bloque de cada característica se use en al menos un test.**

El número de tests es igual al número de bloques de la característica con más bloques

Each Choice Coverage (ECC)

Característica	b ₁	b ₂	b ₃	b ₄
A	A1	A2	A3	A4
B	B1	B2	B3	B4
C	C1	C2	C3	C4

Un valor de cada bloque de cada característica se debe usar en al menos un test.

Requisitos de Test

A1
A2
A3
A4
B1
B2
B3
B4
C1
C2
C3
C4

kindOfTriangle() :

A1, B1, C1
A2, B2, C2
A3, B3, C3
A4, B4, C4

Usando valores:

2, 2, 2
1, 1, 1
0, 0, 0
-1, -1, -1

Todos los bloques usados al menos una vez

¿Cómo elegir combinaciones de valores?

Criterios de combinación de bloques:

- ★ **ACoC** (All combination Coverage): generalmente test suite muy grandes
- ★ **ECC** (Each Choice Coverage): Barato aunque da lugar a pocos test y en general suite de test poco efectivas

¿Cómo elegir combinaciones de valores?

Otros criterios intermedios:

★ **Pair-Wise Coverage (PWC)**: Un requisito de test para cubrir cada par de bloques de diferentes características

El número de tests es al menos el producto de los bloques de la dos características con más bloques.

★ **Puede extenderse a t-Wise Coverage (TWC)**: Un requisito de test para cubrir cada tupla de t bloques de diferentes características

El número de tests es al menos el producto de los bloques de las t características con más bloques.

PWC: Pair-Wise Coverage

kindOfTriangle() : Cada uno Cubre 3 requisitos de tests

A1, B1, C1	A1, B2, C2	A1, B3, C3	A1, B4, C4
A2, B1, C2	A2, B2, C3	A2, B3, C4	A2, B4, C1
A3, B1, C3	A3, B2, C4	A3, B3, C1	A3, B4, C2
A4, B1, C4	A4, B2, C1	A4, B3, C2	A4, B4, C3

Característica	b ₁	b ₂	b ₃	b ₄
A	A1	A2	A3	A4
B	B1	B2	B3	B4
C	C1	C2	C3	C4

Otro ejemplo: Tenemos tres particiones con bloques [A,B], [1,2,3] y [x,y] los tests necesitan cubrir las siguientes 16 requisitos de test.

A,1	B,1	1,x
A,2	B,2	1,y
A,3	B,3	2,x
A,x	B,x	2,y
A,y	B,y	3,x
		3,y

Como cada test
puede cubrir más de
un requisito es
suficiente con los
siguientes 8 tests:

A,1,x	A,2,x	A,3,x
B,1,y	B,2,y	B,3,y
A,1,y	B,1,x	

PWC: Pair-Wise Coverage

kindOfTriangle() : Cada uno Cubre 3 requisitos de tests

A1, B1, C1	A1, B2, C2	A1, B3, C3	A1, B4, C4
A2, B1, C2	A2, B2, C3	A2, B3, C4	A2, B4, C1
A3, B1, C3	A3, B2, C4	A3, B3, C1	A3, B4, C2
A4, B1, C4	A4, B2, C1	A4, B3, C2	A4, B4, C3

Característica	b ₁	b ₂	b ₃	b ₄
A	A1	A2	A3	A4
B	B1	B2	B3	B4
C	C1	C2	C3	C4

Otro ejemplo: Tenemos tres particiones con bloques [A,B], [1,2,3] y [x,y] los tests necesitan cubrir las siguientes 16 requisitos de test.

A ,1	B,1	1 ,x
A,2	B,2	1,y
A,3	B,3	2,x
A ,x	B,x	2,y
A,y	B,y	3,x
		3,y

Como cada test
puede cubrir más de
un requisito es
suficiente con los
siguientes 8 tests:

A,1,x	A,2,x	A,3,x
B,1,y	B,2,y	B,3,y
A,1,y	B,1,x	

PWC: Pair-Wise Coverage

kindOfTriangle() : Cada uno Cubre 3 requisitos de tests

A1, B1, C1	A1, B2, C2	A1, B3, C3	A1, B4, C4
A2, B1, C2	A2, B2, C3	A2, B3, C4	A2, B4, C1
A3, B1, C3	A3, B2, C4	A3, B3, C1	A3, B4, C2
A4, B1, C4	A4, B2, C1	A4, B3, C2	A4, B4, C3

Característica	b ₁	b ₂	b ₃	b ₄
A	A1	A2	A3	A4
B	B1	B2	B3	B4
C	C1	C2	C3	C4

Otro ejemplo: Tenemos tres particiones con bloques [A,B], [1,2,3] y [x,y] los tests necesitan cubrir las siguientes 16 requisitos de test.

A ,1	B,1	1 ,x
A,2	B,2	1,y
A,3	B,3	2,x
A ,x	B,x	2,y
A,y	B,y	3,x
		3,y

Como cada test
puede cubrir más de
un requisito es
suficiente con los
siguientes 8 tests:

<u>A,1,x</u>	A,2,x	A,3,x
B,1,y	B,2,y	B,3,y
A,1,y	B,1,x	

PWC: Pair-Wise Coverage

kindOfTriangle() : Cada uno Cubre 3 requisitos de tests

A1, B1, C1	A1, B2, C2	A1, B3, C3	A1, B4, C4
A2, B1, C2	A2, B2, C3	A2, B3, C4	A2, B4, C1
A3, B1, C3	A3, B2, C4	A3, B3, C1	A3, B4, C2
A4, B1, C4	A4, B2, C1	A4, B3, C2	A4, B4, C3

Característica	b ₁	b ₂	b ₃	b ₄
A	A1	A2	A3	A4
B	B1	B2	B3	B4
C	C1	C2	C3	C4

Otro ejemplo: Tenemos tres particiones con bloques [A,B], [1,2,3] y [x,y] los tests necesitan cubrir las siguientes 16 requisitos de test.

A ,1	B,1	1 ,x
A ,2	B,2	1,y
A,3	B,3	2 ,y
A ,x	B,x	2,y
A,y	B,y	3,x
		3,y

Como cada test
puede cubrir más de
un requisito es
suficiente con los
siguientes 8 tests:

<u>A,1,x</u>	A,2,x	A,3,x
B,1,y	B,2,y	B,3,y
A,1,y	B,1,x	

PWC: Pair-Wise Coverage

kindOfTriangle() : Cada uno Cubre 3 requisitos de tests

A1, B1, C1	A1, B2, C2	A1, B3, C3	A1, B4, C4
A2, B1, C2	A2, B2, C3	A2, B3, C4	A2, B4, C1
A3, B1, C3	A3, B2, C4	A3, B3, C1	A3, B4, C2
A4, B1, C4	A4, B2, C1	A4, B3, C2	A4, B4, C3

Característica	b ₁	b ₂	b ₃	b ₄
A	A1	A2	A3	A4
B	B1	B2	B3	B4
C	C1	C2	C3	C4

Otro ejemplo: Tenemos tres particiones con bloques [A,B], [1,2,3] y [x,y] los tests necesitan cubrir las siguientes 16 requisitos de test.

A ,1	B,1	1, x
<u>A</u> ,2	B,2	1,y
A,3	B,3	2, <u>y</u>
A ,x	B,x	2,y
A,y	B,y	3,x
		3,y

Como cada test
puede cubrir más de
un requisito es
suficiente con los
siguientes 8 tests:

<u>A</u> ,1,x	<u>A</u> ,2,x	A,3,x
B,1,y	B,2,y	B,3,y
A,1,y	B,1,x	

Base Choice Coverage (BCC)

Para cada característica se designa un bloque como **elección base**. Un test base se conforma usando la elección base de cada característica. Los siguientes tests se forman manteniendo todos los valores constantes (bases) menos uno y usando cada elección no base en las demás características.

El número de tests es igual a un test base más un test por cada uno de los restantes bloques.

Base Choice Coverage (BCC)

Característica	b ₁	b ₂	b ₃	b ₄
A	A1	A2	A3	A4
B	B1	B2	B3	B4
C	C1	C2	C3	C4

Base Choice Coverage (BCC)

Elección de bloques base

Característica	b_1	b_2	b_3	b_4
A	A1	A2	A3	A4
B	B1	B2	B3	B4
C	C1	C2	C3	C4

Base Choice Coverage (BCC)

Elección de bloques base

Característica	b ₁	b ₂	b ₃	b ₄
A	A1	A2	A3	A4
B	B1	B2	B3	B4
C	C1	C2	C3	C4

kindOfTriangle() :

A1, B1, C1	A1, B1, C2	A1, B2, C1	A2, B1, C1
	A1, B1, C3	A1, B3, C1	A3, B1, C1
	A1, B1, C4	A1, B4, C1	A4, B1, C1

Base Choice Coverage (BCC)

Elección de bloques base

Característica	b ₁	b ₂	b ₃	b ₄
A	A1	A2	A3	A4
B	B1	B2	B3	B4
C	C1	C2	C3	C4

kindOfTriangle() :

A1, B1, C1	A1, B1, C2	A1, B2, C1	A2, B1, C1
	A1, B1, C3	A1, B3, C1	A3, B1, C1
	A1, B1, C4	A1, B4, C1	A4, B1, C1

Base Choice Coverage (BCC)

Elección de bloques base

Característica	b ₁	b ₂	b ₃	b ₄
A	A1	A2	A3	A4
B	B1	B2	B3	B4
C	C1	C2	C3	C4

kindOfTriangle() :

A1, B1, C1	A1, B1, C2	A1, B2, C1	A2, B1, C1
	A1, B1, C3	A1, B3, C1	A3, B1, C1
	A1, B1, C4	A1, B4, C1	A4, B1, C1

Base Choice Coverage (BCC)

Elección de bloques base

Característica	b ₁	b ₂	b ₃	b ₄
A	A1	A2	A3	A4
B	B1	B2	B3	B4
C	C1	C2	C3	C4

kindOfTriangle() :

A1, B1, C1	A1, B1, C2	A1, B2, C1	A2, B1, C1
	A1, B1, C3	A1, B3, C1	A3, B1, C1
	A1, B1, C4	A1, B4, C1	A4, B1, C1

Base Choice Coverage (BCC)

El **test base** debe ser viable, es decir, las elecciones deben ser compatibles.

Tests que siguen un camino feliz suelen ser buenas elecciones.

La elección base es una decisión de diseño crucial. Los diseñadores de tests deberían documentar sus decisiones en este sentido.

Restricciones entre características

Hay dos tipos de restricciones:

- ★ Un bloque de una característica no se puede combinar con un bloque específico de otra.
- ★ Un bloque de una característica puede combinarse SOLO con un bloque específico de otra.
- ★ La manipulación de restricciones depende del criterio considerado:
 - ★ **ACoC, PWC**: Eliminar las combinaciones no viables.
 - ★ **BCC**: Cambiar un valor por otro que no sea base para encontrar una combinación viable.

Restricciones entre características

```
public int numberOfOccurrences(List, element);  
//Effects: if list or element is Null throws ValueError  
//else return number of occurrences of element in List
```

Característica	B1	B2	B3
A: ocurrencia del elemento en list	no aparece	aparece una vez	aparece más de una vez
B : primero en la lista	true	false	
C: último en la lista	true	false	
Combinaciones inválidas: (A.B1, B.B1), (A.B1 C.B1)			

restricciones:

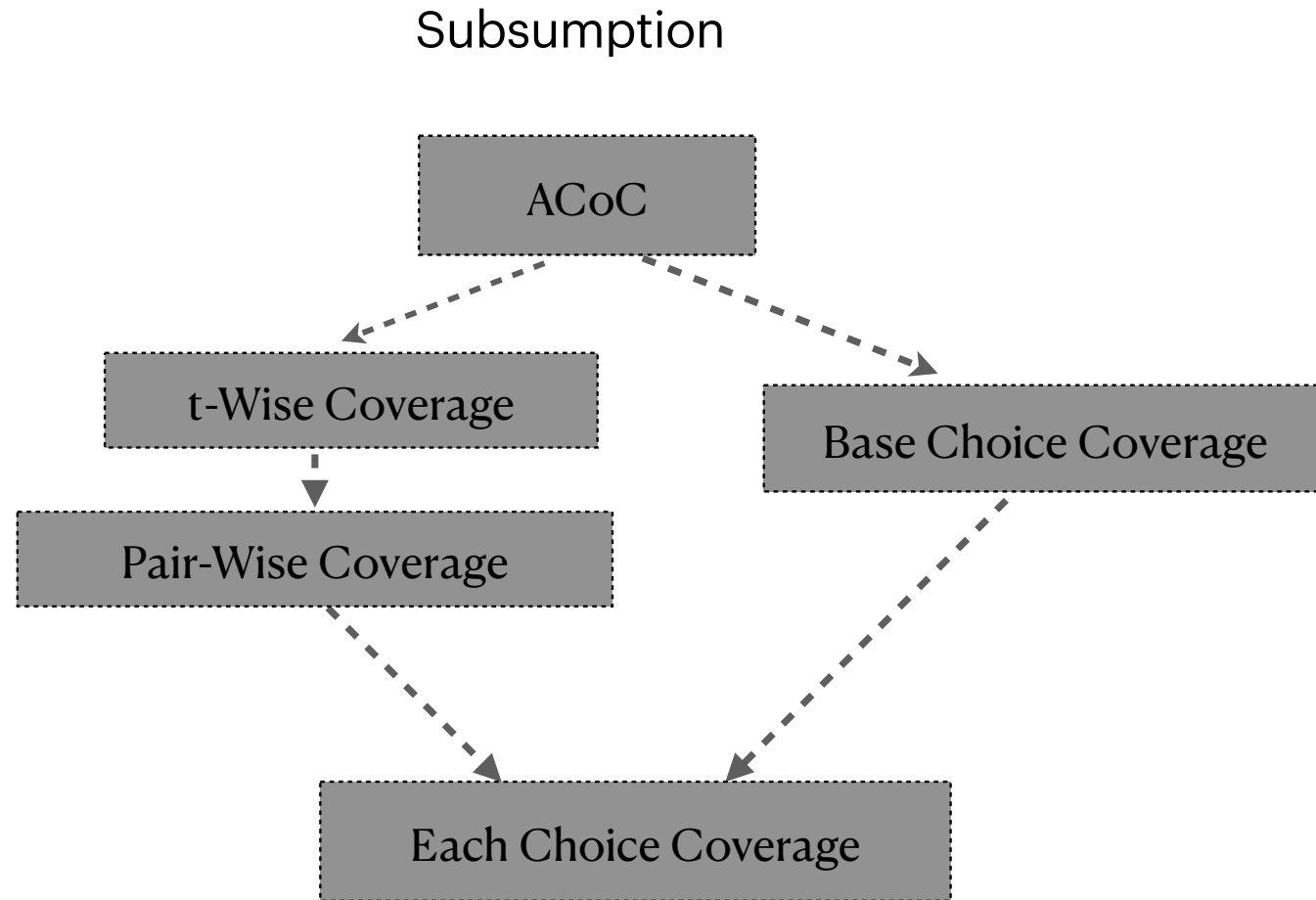
!(No aparece && primero en la lista = true)

!(No aparece && último en la lista = true)

Resumen del particionado del espacio de entradas

- Aplicable a todos los niveles de testing unidad, módulo, integración, sistema, aceptación etc.
- Basado sólo en el espacio de entradas del programa y en sus especificaciones, no en la implementación
- Fácil de adaptar para requerir más o menos esfuerzo de testing
- Ampliamente usado

Crterios: Relación entre ellos



Lectura recomendada

- ★ <https://www.nist.gov/programs-projects/automated-combinatorial-testing-software-acts>
- ★ Paul Ammann, Jeff Offutt, Introduction to Software Testing, Cap 6